

Pan-European University, Faculty of Informatics

Interim Research Report

Translation of subset of BPMN to a low-code language based on object-centric processes

of the project

**Requirements and formal definition of a low-code
language based on object-centric processes -
LowcodeOCP**

Authors:

Gabriel Juhás, Milan Mladoniczky, Juraj Mažári, Tomáš Kováčik and
Michal Drobný

Contact:

gabriel.juhas@paneurouni.com, milan.mladoniczky@paneurouni.com

Funded by the EU NextGenerationEU through the Recovery and Resilience Plan for Slovakia
under the project No.

09I03-03-V04-00493

May 15, 2026

Version: v0.1

Abstract

This report addresses the gap between business process models and executable applications by transforming BPMN diagrams into Petriflow classes. By focusing on the empirically validated core subset of BPMN, the proposed framework transforms high-level designs into prescriptive source code where process instances can act as reactive, data-carrying objects. Unlike low-level Petri nets, Petriflow utilizes variable weight arcs and integrated data attributes to model complex patterns, such as inclusive OR-gateways, more intuitively and with fewer elements. This approach can also transform elements such as swimlanes, data objects, loops and sub-processes directly into Petriflow constructs or resources.

Contents

Contents	i
List of Figures	ii
1 Introduction	1
2 Petriflow	2
3 Workflow Transformation	3
3.1 Basic Elements	3
3.2 Gateways	4
3.3 Participants	6
3.4 Data	6
3.5 Loops	7
3.6 Sub-processes	7
4 Future Work	9
5 Conclusion	10

List of Figures

3.1	The <i>Request</i> process in form of a BPMN diagram.	4
3.2	Overview of core BPMN elements and resulting Petriflow constructs. . .	4
3.3	Transformation of the Start Event and first two Activities, <i>Create a new request</i> and <i>Register request</i> , of the <i>request</i> process.	4
3.4	Example of a transformation of a BPMN gateway into Petriflow. In this configuration it represent an inclusive <i>OR-gateway</i> with A and C as active paths.	5
3.5	The <i>Request</i> process transformed into a Petriflow class.	6
3.6	Examples of <i>do-while</i> and <i>while-do</i> loops.	7

Chapter 1

Introduction

In the current industrial landscape, the Business Process Modeling Notation (BPMN) is the standard for graphical representation of workflows, while separate standards like entity–relationship models or UML class diagrams model data that are used in workflows or support them. This approach often results in a gap between models and applications, because BPMN elements lack formal execution semantics [DDO08], [Woh+06].

This report proposes transformation of subset of BPMN elements into a unified, executable framework based on the Petriflow language. By transforming the *core* subset of BPMN, elements that are most frequently used in practice [MR08], into *Petriflow classes*, we can move beyond purely descriptive models. In this framework, data objects and sub-processes can be treated as independent *Petriflow classes* that function as the source code of the system, where process instances act as data-carrying objects and references, compositions, and interactions of instances are managed via data attributes.

Chapter 2

Petriflow

Process-centric objects (PCO) are the core building blocks of the Petriflow language. Each PCO is defined by its *lifecycle*, *data attributes*, and *interfaces*. The *lifecycle* of a PCO is represented by place/transition nets [DJ01; Des05; DR15], extended with read [Vog02], inhibitor [AJ15], and reset arcs [Aal+09]. Inspired by self-modifying nets [Val78] it also allows to use variable weight of arcs, given by values of data attributes (that can be cast or evaluated into a non-negative integer) or markings of places. Transitions of the resulting nets represents task or activities, executed manually by users or automatically by the system, with interval semantics and *assign* (consumes tokens from inputs), *finish* (produces tokens to outputs), and *cancel* (returns consumed tokens to inputs) events. *Data attributes* are objects with a given type, value of that type, id, title, validations, etc. (see petriflow.org for more details). *Interfaces* are represented as forms, where data fields are objects with a reference to a data attribute, its accessibility (visible, editable, hidden), if its required to have a value, etc.

Chapter 3

Workflow Transformation

While the BPMN specification defines over 50 modeling constructs, empirical research suggests that only 20% of those constructs are regularly used in practice [MR08]. This indicates that a transformation focused on a small, core set of elements is sufficient to translate a significant portion of existing BPMN models. While numerous established methods exist for mapping BPMN to Petri nets [DDO08; Wes19], these transformations often result in complex models when attempting to represent data-controlled logic, like inclusive OR-gateways, or synchronization. In contrast, transforming BPMN to Petriflow can yield simpler and more understandable results because Petriflow extends the standard Petri net formalism with arcs with variable weights and integrated data attributes, allowing it to capture complex workflow patterns more intuitively and with fewer elements than its low-level Petri net counterparts.

We use the generalized *Request* process in figure 3.1 to demonstrate this. In this process a *Customer* can file a request (*Create a new request* activity) that is then registered (*Register request* activity) by an employee of *Registration*, who decides (*Relevant departments?* gateway) which departments are responsible for processing of the request - *Legal*, *Technical*, or *HR*. After all responsible departments process the request the registration employee writes a reply for the customer (*Reply to customer* activity).

3.1 Basic Elements

In figure 3.2 we provide a brief overview of transformations of the basic elements that result in Petriflow constructs. The *Start event* is transformed into a task **start** with an input place **in** with one token, so that the task is enabled. The *End event* is transformed into a task **end** with an output place **out**. Both *Activity* and *Intermediate event* are transformed into a task, with input and output places shared with previous and following activities or events. Labels of the BPMN elements are also copied to corresponding Petriflow tasks.

Using these transformations the first part of the request process (until OR-split) can

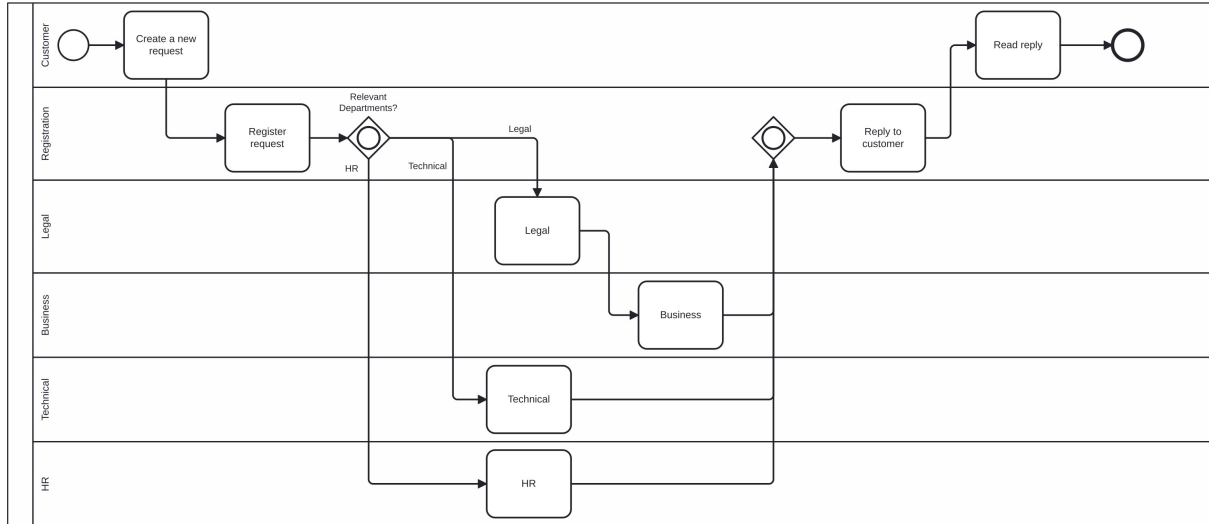
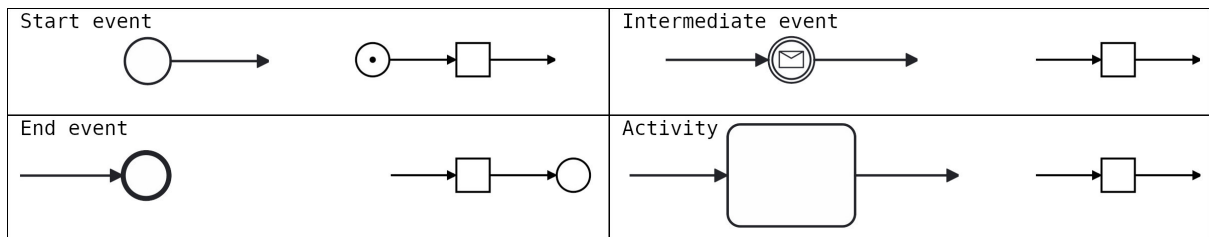
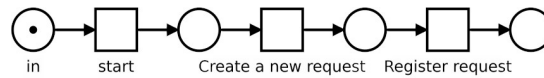
Figure 3.1: The *Request* process in form of a BPMN diagram.

Figure 3.2: Overview of core BPMN elements and resulting Petriflow constructs.

be transformed into Petriflow class in Figure 3.3.

Figure 3.3: Transformation of the Start Event and first two Activities, *Create a new request* and *Register request*, of the *request* process.

Tasks for start and end events could be merged with following or previous tasks to simplify the workflow and reduce the number of tasks, but having a separate task has a distinct advantage in Petriflow. For example consider an example when one process needs to start another process with some input data. In Petriflow creating an instance and setting data are two separate events: `createCase` and `setData`. Following this principle we can use an action that automatically creates the new instance, assigns the `start` task, sets values of data fields in the task form, and finishes the task.

3.2 Gateways

Both AND and XOR gateways are easily transformed into Petri nets as can be seen in [DDO08]. In contrast to transform OR gateway with 3 possible paths we need 7 transitions

to represent each possibility. In general the number of transition we need to model OR gateways with n paths is $2^n - 1$.

All types of BPMN gateways can be transformed into Petriflow using a singular construct (see Figure 3.4. The gateway split can be transformed into a transition with output places representing each path of the gateway. Each output place is connected with a variable weight arc referencing a separate data attribute, that determines if the path will be taken. Another decision data attribute with options is added, options represent each of the paths and change in value of this data attribute triggers the change of value of data attributes referenced by variable arcs.

- **AND-gateway** - all data attributes has value equal to one, meaning that every path will be active, and the value cannot be changed.
- **XOR-gateway** - exactly one data attribute has value equal to one and others are equal to zero, meaning that only one path will be active, the decision data attribute is of type enumeration, which can have only one option selected to satisfy the condition.
- **OR-gateway** - at least one data attribute has value equal to one, meaning that any combination of paths will be active, the decision data attribute is of type multichoice, which can have multiple options selected, and is set as required so at least one option must be selected to satisfy the condition.

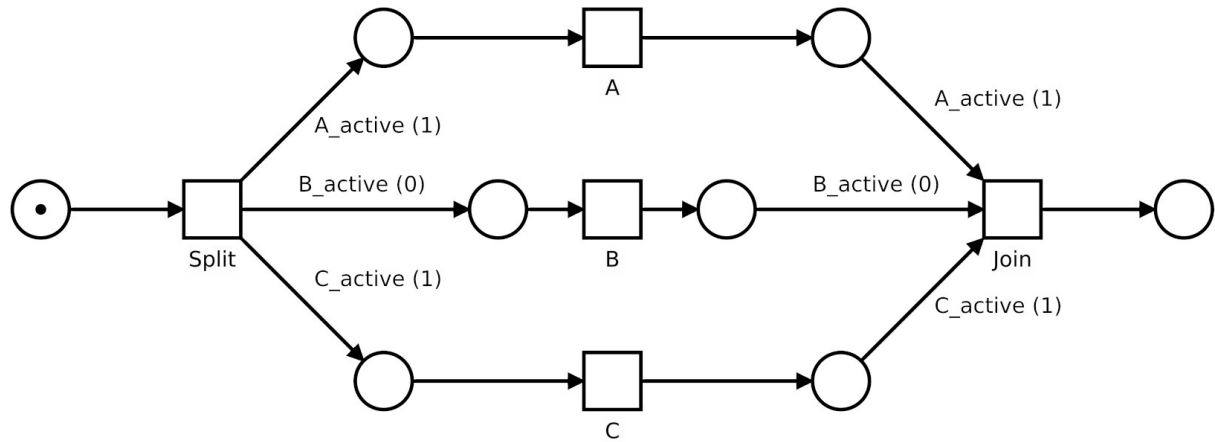


Figure 3.4: Example of a transformation of a BPMN gateway into Petriflow. In this configuration it represent an inclusive *OR-gateway* with A and C as active paths.

Together with previously defined transformation rules the request process can be transformed into a Petriflow class shown in figure 3.5.

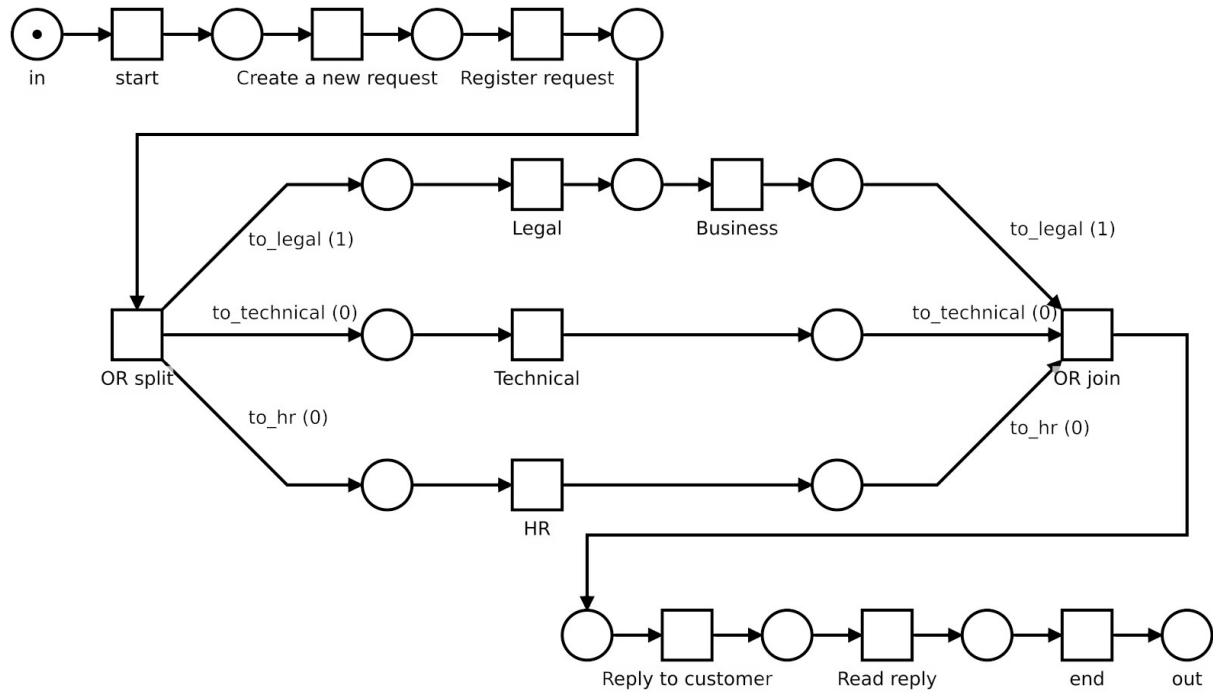


Figure 3.5: The *Request* process transformed into a Petriflow class.

3.3 Participants

Participants of the process in form of **Pools** and **Swimlanes** can be directly transformed into process roles defined in the Petriflow language. For each activity present in a *Swimlane* a role reference (`roleRef`) is added to the corresponding transition in the Petriflow class with a `perform` permission.

In our example process swimlanes *Customer*, *Registration*, *Legal*, *Business*, *Technical*, and *HR* are transformed into roles of the *Request* process. Role references are created for tasks in corresponding lanes, e.g. tasks *Create a new request* and *Read reply* have a `roleRef` of *Customer* and permission *perform*. Also a role *System* is added to handle the automatic execution of the OR-split and OR-join related tasks by the system.

3.4 Data

Data related elements like **Data Object** and **Data Store** can be transformed if they reference a model of the data objects in form of a ER diagram or UML Class diagram. The *Data Store* element could be transformed into an automatic task with actions that will handle read/write operations on external data store. The *Data Object* element could be transformed in two ways - as integral data attributes of the Petriflow class, or as a separate Petriflow class and only referenced using a case or task reference data attribute.

Elements **Data Input** and **Data Output** could be understood as *setters* and *getters* on a set of data attributes of the Petriflow class and modeled accordingly with *Data Input*

task having editable (and possibly required) data fields and *Data Output* task having only visible data fields.

3.5 Loops

In BPMN activities can have loop markers that denotes *do-while* or *while-do* loops. The **do-while** loop of a BPMN task can be modeled using a control transition, following the task, with two outgoing arcs with variable weights as seen in figure 3.6. The arc responsible for looping leads back to the input place of the task and references data attribute `do_while_true` with default value 1. Another arc leads to place connecting it to the following task of the original flow and references data attribute `do_while_false` with default value 0. The control task contains pre-finish action that checks the loop condition and if the loop should break it sets the value of `do_while_true` to 0 and `do_while_false` to 1, ensuring that finishing the control task will break the loop and will continue with the original flow of the process.

In similar manner the **while-do** loop is modeled. In this case the control transition is modeled before the task itself. The arc responsible for looping leads from the control transition to the input place of the task and references data attribute `while_do_true` with default value 1. The arc responsible for loop breaking leads from the control task to the place connecting it to the following task of the original flow and references data attribute `while_do_false` with default value 0. The control transition contains the same action as in *do-while* loop, that sets value of `while_do_true` to 0 and `while_do_false` to 1 if the loop should break.

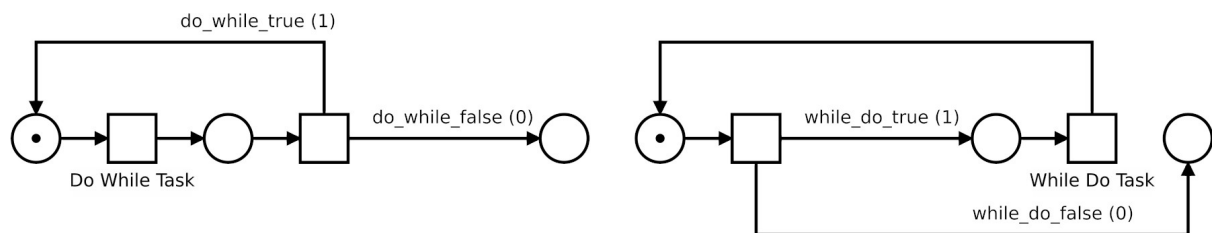


Figure 3.6: Examples of *do-while* and *while-do* loops.

3.6 Sub-processes

Petriflow does not define sub-processes in the same way as BPMN, but rather uses references to create interaction and composition between instances of Petriflow classes. In this way we can transform sub-process activity in BPMN diagram into a Petriflow task **sub-process** which when started will trigger the `createCase` event of a specific Petriflow class, as mention in section 3.1, and wait for the child process to finish. At least the

reference to the **sub-process** task is passed to the child object using the **setData** event. After the child object workflow is finished, it triggers **finishTask** event on the referenced **sub-process**, so the parent workflow can continue.

Chapter 4

Future Work

In the BPMN standard, data is primarily handled through descriptive elements such as Data Objects or Data Stores. Consequently, in industry practice, domain data structures are rarely modeled within the workflow itself. Separate standards like ER diagrams or UML class diagrams are used to define the system's data model. Since Petriflow unifies workflows and data attributes into objects with lifecycle, we could translate ER or UML class diagrams directly into Petriflow classes and use data attributes of type `caseRef` to model their relative references, compositions, and interactions.

With the rise of AI tools, we are also exploring possible use cases for AI agents to help developers create Petriflow-based applications. Most free to use AI tools struggle to create Petriflow processes without any persistent instructions and context. With the use of well-structured markdown instructions, based on the transformation rules defined in this report, we got promising early results with both Claude AI and Gemini. Both AI tools transformed the *request* process in the form of a BPMN diagram into a functional Petriflow process with workflow, roles, data attributes and actions, with some degree of errors that came mainly from inconsistencies in the persistent context file.

Chapter 5

Conclusion

This report describes a transformation of the core BPMN subset into executable Petriflow classes. By leveraging the unique features of the Petriflow language (variable weights of arcs, integrated data attributes) we can bypass the complexities typically associated with mapping BPMN to low-level Petri nets. Transforming BPMN into Petriflow classes provides a formal and directly executable foundation for process-driven applications. Future transformations of domain datamodels and integration with AI agents shows further promise in automated transformation of high-level design into functional application.

Bibliography

- [Aal+09] Wil M. P. van der Aalst et al. “Soundness of Workflow Nets with Reset Arcs”. In: *Trans. Petri Nets Other Model. Concurr.* 3 (2009), pp. 50–70.
- [AJ15] Mohammed A. Alqarni and Ryszard Janicki. “On Interval Process Semantics of Petri Nets with Inhibitor Arcs”. In: *Petri Nets*. Vol. 9115. Lecture Notes in Computer Science. Springer, 2015, pp. 77–97.
- [Des05] Jörg Desel. “Process Modeling Using Petri Nets”. In: *Process-Aware Information Systems*. Wiley, 2005, pp. 147–177.
- [DJ01] Jörg Desel and Gabriel Juhás. “”What Is a Petri Net?”” In: *Unifying Petri Nets*. Vol. 2128. Lecture Notes in Computer Science. Springer, 2001, pp. 1–25.
- [DR15] Jörg Desel and Wolfgang Reisig. “The concepts of Petri nets”. In: *Softw. Syst. Model.* 14.2 (2015), pp. 669–683.
- [DDO08] Remco M. Dijkman, Marlon Dumas, and Chun Ouyang. “Semantics and analysis of business process models in BPMN”. In: *Information and Software Technology* 50.12 (2008), pp. 1281–1294. ISSN: 0950-5849. DOI: <https://doi.org/10.1016/j.infsof.2008.02.006>. URL: <https://www.sciencedirect.com/science/article/pii/S0950584908000323>.
- [MR08] Michael zur Muehlen and Jan Recker. “How Much Language Is Enough? Theoretical and Practical Use of the Business Process Modeling Notation”. In: *Advanced Information Systems Engineering*. Ed. by Zohra Bellahsene and Michel Léonard. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 465–479. ISBN: 978-3-540-69534-9.
- [Val78] Rüdiger Valk. “Self-Modifying Nets, a Natural Extension of Petri Nets”. In: *ICALP*. Vol. 62. Lecture Notes in Computer Science. Springer, 1978, pp. 464–476.
- [Vog02] Walter Vogler. “Partial order semantics and read arcs”. In: *Theor. Comput. Sci.* 286.1 (2002), pp. 33–63.
- [Wes19] Mathias Weske. *Mapping BPMN to Petri Nets*. Hasso Plattner Institute. 2019. URL: <https://open.hpi.de/courses/bpm2019/items/5dnmhJUCSwAsUMi1TUowBr> (visited on 04/10/2026).

- [Woh+06] P. Wohed et al. “On the Suitability of BPMN for Business Process Modelling”. In: *Business Process Management*. Ed. by Schahram Dustdar, José Luiz Fiadeiro, and Amit P. Sheth. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 161–176. ISBN: 978-3-540-38903-3.